

Compilador JSS

Documentacao do Front-end - Explicacao das Classes

Analise lexica, sintatica e semantica da linguagem Java Script Simplificado (JSS), implementada em Java com ANTLR 4. Este documento explica, de forma didatica, cada classe do codigo e o conceito de Compiladores que ela representa.

Disciplina de Compiladores - UFPI - 2026.1

Sumario

1. Visao geral e arquitetura
2. A gramatica JSS.g4 (analise lexica e sintatica)
3. Classes geradas pelo ANTLR
4. Pacote erro (tratamento de erros)
5. Pacote tipos (sistema de tipos)
6. Pacote simbolos (tabela de simbolos e escopo)
7. Pacote semantico (analise semantica)
8. Orquestracao (Main e Compilador)
9. Pacote codegen (gancho do back-end Jasmin)
10. Mapa: classe -> fase do compilador
11. Roteiro de apresentacao

1. Visao geral e arquitetura

O compilador lê o código-fonte JSS pela **entrada padrão** e executa as três fases clássicas do **front-end** de um compilador, parando no **primeiro erro** encontrado. A saída é uma única mensagem: Linguagem compilada com sucesso ou Erro na linha X: descrição do erro.

```
Entrada padrao (programa.jss, em UTF-8)
  |
  v
[1] Analise lexica      -> JSSLexer          : texto -> tokens
  |
  v
[2] Analise sintatica  -> JSSParser         : tokens -> arvore sintatica
  |
  v
[3] Analise semantica  -> AnalisadorSemantico : valida tipos, escopos e regras
  |
  v
Saída: "Linguagem compilada com sucesso" ou "Erro na linha X: ..."
```

Duas ideias centrais sustentam o projeto:

- **ANTLR 4 + gramática.** A partir de um único arquivo de gramática (JSS.g4), o ANTLR **gera** automaticamente o analisador léxico e o sintático. Usamos o algoritmo **ALL(*)**, que resolve conflitos pela **ordem** das alternativas (não exige uma gramática provadamente não-ambígua).
- **Padrão Visitor.** A análise semântica é um **visitor** que percorre a árvore sintática gerada, calculando o tipo de cada expressão e validando as regras da linguagem.

Organização do código (uma classe por arquivo, pacotes por responsabilidade):

- erro/ - representação e reporte de erros;
- tipos/ - o sistema de tipos;
- simbolos/ - a tabela de símbolos e os escopos;
- semantico/ - a análise semântica (o núcleo da validação);
- codegen/ - o gancho para o back-end (gerador Jasmin), ainda não implementado;
- Main e Compilador - a entrada e a orquestração das fases.

2. A gramatica JSS.g4 (analise lexica e sintatica)

JSS.g4 (src/main/antlr4)

Responsabilidade. A especificacao formal da linguagem JSS numa gramatica combinada (lexico + sintaxe), da qual o ANTLR gera o lexer e o parser. Sao 158 linhas escritas a mao que substituem milhares de linhas de codigo gerado.

- **Lexico:** define os tokens - 21 palavras reservadas, IDENTIFIER, literais (int, real com expoente, str com escapes, bool), operadores, e o comentario de linha // (descartado);
- **Sintaxe:** regras para programa, declaracoes (let/const), funcoes, classes, comandos (if/while/for) e expressoes;
- **Precedencia:** fica toda em **uma** regra expr recursiva a esquerda, com as alternativas ordenadas da maior para a menor precedencia, e <assoc=right> em ** e nas atribuicoes - exatamente a Tabela 1 da especificacao.

Conceito de Compiladores: Gramatica livre de contexto + definicoes lexicas: a base formal das analises lexica e sintatica.

Como explicar: E a "regra do jogo" da linguagem; eu escrevo as regras e o ANTLR constroi o lexer e o parser inteiros a partir delas.

Trecho da regra de expressoes (a ordem das alternativas define a precedencia):

```
expr
: expr '[' expr ']'           # indexExpr    // mais alta
| expr '.' IDENTIFIER '('(' argList? ')')? # memberExpr // attr/metodo; console.log
| op=('!'|'|'|'-'|'|'++'|'|'--') expr      # unaryExpr
| <assoc=right> expr '**' expr             # powExpr
| expr op=('*'|'|'|'%' ) expr              # mulExpr
| expr op=('+'|'|'-' ) expr                # addExpr
| expr op=('<'|'|'>'|'|'<='|'|'>=') expr    # relExpr
| expr op=('=='|'|'!=') expr              # eqExpr
| expr '&&' expr                          # andExpr
| expr '||' expr                         # orExpr
| <assoc=right> expr op=('='|'|'+=') ... ) expr # assignExpr // mais baixa
| primary                                # primaryExpr
;
```

3. Classes geradas pelo ANTLR

Estas quatro classes **nao sao escritas a mao**: o ANTLR as gera (cerca de 4500 linhas) a partir da JSS.g4, em target/generated-sources, e elas sao recriadas a cada build. Por isso nao ficam no controle de versao.

JSSLexer *(gerada pelo ANTLR)*

Responsabilidade. O analisador lexico. Transforma o texto-fonte numa sequencia de **tokens** (palavras-chave, identificadores, numeros, strings, operadores).

Conceito de Compiladores: Analise lexica (scanning): de caracteres para tokens.

Como explicar: E o "leitor de palavras": quebra o texto nas pecas minimas que o parser entende.

JSSParser *(gerada pelo ANTLR)*

Responsabilidade. O analisador sintatico. Usando o algoritmo **ALL(*)**, monta a **arvore sintatica** (parse tree) a partir dos tokens, conforme as regras da gramatica.

Conceito de Compiladores: Analise sintatica (parsing): de tokens para arvore sintatica.

Como explicar: E o "montador da estrutura": confere se a sequencia de tokens forma um programa valido e cria a arvore.

JSSBaseVisitor / JSSVisitor *(gerada pelo ANTLR)*

Responsabilidade. A infraestrutura do padrao **Visitor**. JSSVisitor e a interface (um metodo por regra); JSSBaseVisitor e a implementacao base que percorre a arvore. Nosso AnalisadorSemantico herda dela e sobrescreve apenas os metodos que importam.

Conceito de Compiladores: Padrao de projeto Visitor aplicado a percorrer a arvore sintatica.

Como explicar: E o "esqueleto de passeio" pela arvore; eu so preencho as visitas que preciso validar.

4. Pacote erro (tratamento de erros)

ErroCompilacao (erro)

Responsabilidade. Execução que representa qualquer erro de compilação (lexico, sintatico ou semantico) e guarda a **linha** onde ele ocorreu.

- Estende RuntimeException (nao checada) para poder ser lancada de dentro do ANTLR e do visitor;
- mensagemFormatada() devolve exatamente Erro na linha X: descricao - o formato exigido pela especificacao.

Conceito de Compiladores: Relato de erros: o compilador precisa informar o usuario com mensagem e localizacao precisas.

Como explicar: E a "mensagem de erro" do compilador: carrega a linha e o texto, e sabe se formatar no padrao pedido.

OuvinteErroSintatico (erro)

Responsabilidade. Ouvinte de erros do ANTLR que **interrompe a compilação no primeiro erro** lexico ou sintatico.

- Estende BaseErrorListener; o metodo syntaxError e chamado pelo lexer e pelo parser quando algo nao casa;
- Lanca ErroCompilacao na 1a ocorrencia (regra "parar no primeiro erro") e traduz a mensagem para portugues;
- E registrado no lexer e no parser, substituindo o ouvinte padrao do ANTLR.

Conceito de Compiladores: Tratamento de erros lexicos e sintaticos durante a tokenizacao e o parsing.

Como explicar: Sempre que o lexer ou o parser acha algo invalido, ele me avisa e eu paro tudo na hora, dizendo a linha.

5. Pacote tipos (sistema de tipos)

Tipo (tipos)

Responsabilidade. A interface base de todo o sistema de tipos do JSS.

- Implementada por TipoPrimitivo, TipoVetor e TipoClasse;
- Define nome() - a representacao textual usada nas mensagens (ex.: int[][]).

Conceito de Compiladores: Sistema de tipos: a abstracao comum que permite tratar todos os tipos de forma uniforme.

Como explicar: E o "contrato" que todo tipo cumpre; o resto do compilador fala com tipos por esta interface.

TipoPrimitivo (tipos)

Responsabilidade. Os tipos basicos da linguagem, como um enum: int, real, str, bool, alem de void e null.

- Sendo enum, cada tipo e unico (comparavel por ==) e suporta switch;
- porNome("int") converte o texto da gramatica no tipo correspondente.

Conceito de Compiladores: Tipos primitivos do sistema de tipos.

Como explicar: Sao os tipos "de fabrica" da linguagem; uso um enum porque o conjunto e fixo e fechado.

TipoVetor (tipos)

Responsabilidade. O tipo dos vetores. Vetores **multidimensionais** sao representados por **aninhamento**.

- int[3][4] e TipoVetor(TipoVetor(int,4),3); numDimensoes() conta os niveis e tipoBase() chega ao escalar;
- getElemento() devolve o tipo apos indexar uma vez (m[i]);
- A igualdade considera tipo-base + numero de dimensoes (ignora o tamanho de cada dimensao).

Conceito de Compiladores: Tipos compostos/derivados e como representa-los recursivamente.

Como explicar: Um vetor 2D e "um vetor de vetores"; por isso a classe se aninha, e cada [] remove uma camada.

TipoClasse (tipos)

Responsabilidade. O tipo de um objeto, identificado pelo **nome** da classe.

- Guarda so a identidade (nome); a estrutura detalhada (atributos/metodos) fica em SimboloClasse;
- Mantem a camada tipos sem depender de simbolos (evita dependencia circular).

Conceito de Compiladores: Tipos definidos pelo usuario (classes).

Como explicar: Quando declaro let Ponto p, o tipo de p e este TipoClasse("Ponto").

Tipos (tipos)

Responsabilidade. Utilitarios com as **regras de compatibilidade** entre tipos.

- ehNumerico, ehPrimitivo; promoverNumerico(a,b) (real se algum for real);
- compativelAtribuicao: permite int->real, rejeita real->int, aceita null->objeto;
- comparavelIgualdade para == e !=.

Conceito de Compiladores: Regras de compatibilidade e coercao (promocao) do sistema de tipos.

Como explicar: E onde moram as regras de "combinar tipos": posso por um int onde se espera real, mas nao o contrario.

6. Pacote simbolos (tabela de simbolos e escopo)

Simbolo (simbolos)

Responsabilidade. Uma entrada da tabela de simbolos: um identificador declarado, com seu tipo e metadados.

- Guarda nome, tipo, se e constante (const) ou mutavel, se foi inicializado e a linha;
- Funcoes e classes tambem sao Simbolo (subclasses), para compartilharem o mesmo espaco de nomes.

Conceito de Compiladores: Tabela de simbolos: o registro de cada identificador declarado.

Como explicar: Cada variavel/constante/funcao/classe vira um "cadastro" destes, com tipo e se pode ser alterada.

SimboloFuncao (simbolos)

Responsabilidade. Simbolo de uma funcao (ou metodo): o tipo de **retorno** e os tipos dos **parametros**.

- Usado para validar chamadas: confere a quantidade (aridade) e os tipos dos argumentos;
- E subclasse de Simbolo, entao o nome da funcao nao pode colidir com variaveis/constantes.

Conceito de Compiladores: Assinaturas de funcoes na tabela de simbolos.

Como explicar: Guarda a "assinatura" da funcao para eu checar se a chamada passou os argumentos certos.

SimboloClasse (simbolos)

Responsabilidade. Simbolo de uma classe: guarda o **layout** - atributos, metodos e parametros do construtor.

- Atributos e metodos em LinkedHashMap (preserva a ordem de declaracao, util ao back-end);
- Usado para validar new, acesso a atributos (obj.x) e chamadas de metodo (obj.m()).

Conceito de Compiladores: Tipos estruturados definidos pelo usuario, na tabela de simbolos.

Como explicar: E a "planta" da classe: quais atributos e metodos ela tem, para validar o uso de objetos.

Escopo (simbolos)

Responsabilidade. Um nivel de escopo: um mapa nome-para-simbolo com referencia ao escopo pai.

- O escopo global tem pai nulo; blocos (funcao, if, while, for) encadeiam pelo pai;
- Permite buscar do mais interno ao mais externo e o **shadowing**.

Conceito de Compiladores: Escopo lexico: a visibilidade dos identificadores.

Como explicar: Cada par de chaves { } cria um destes; ele sabe quem e o escopo "de fora" dele.

TabelaDeSimbolos (simbolos)

Responsabilidade. A **pilha de escopos** usada durante a analise semantica.

- declarar() verifica **so o escopo atual** - redeclarar no mesmo escopo e erro;
- resolver() busca do interno ao externo - por isso o shadowing e permitido;
- entrarEscopo() / sairEscopo() ao abrir e fechar blocos.

Conceito de Compiladores: Tabela de simbolos com regras de escopo e shadowing.

Como explicar: E a memoria do analisador: ao entrar num bloco eu empilho um escopo, ao sair eu desempilho; assim sei o que esta visivel.

7. Pacote semantico (analise semantica)

AnalizadorSemantico *(semantico)*

Responsabilidade. O **nucleo** da analise semantica: percorre a arvore sintatica (padrao **Visitor**) validando todas as regras da linguagem.

- Estende `JSSBaseVisitor<Tipo>`: cada metodo `visitX` devolve o **tipo** da expressao; comandos retornam `null`;
- Valida escopos, tipos de operadores, atribuicoes, `const`, `++/--`, funcoes (so chamaveis **depois** de declaradas) e recursao, nativas, vetores/multidim e classes (`this`, `new`, atributos, metodos);
- Para no **primeiro erro**, lancando `ErroCompilacao` com a linha (`ctx.getStart().getLine()`);
- Sobrescreve `visit()` num unico ponto para **anotar** cada expressao com seu tipo (reuso pelo back-end).

Conceito de Compiladores: Analise semantica: verificacao de tipos e das regras de contexto que a gramatica nao captura.

Como explicar: E o "professor rigoroso" do compilador: a sintaxe pode estar certa, mas ele confere se faz sentido (os tipos batem? a variavel existe? a funcao foi declarada?).

RegrasOperadores *(semantico)*

Responsabilidade. Concentra as **regras de tipo** de cada operador aritmetico, relacional e logico.

- `binario(op, a, b)` e `unario(op, t)` devolvem o tipo do resultado ou lancam erro;
- Ex.: `+` entre `int` e `real` -> `real`; `%` e `**` so entre `int`; relacionais so entre numericos; `&&/|` so entre `bool`.

Conceito de Compiladores: Regras de tipagem de expressoes (type checking).

Como explicar: E a tabela de "o que cada operador aceita e o que devolve"; separei numa classe so para ficar legivel.

ValidadorNativas *(semantico)*

Responsabilidade. As **regras de tipo das funcoes nativas**: `input`, `console.log` e os casts.

- `console.log` aceita `int/real/str/bool`; `input` aceita so `int/real/str` (`bool` nao);
- `validarCast`: qualquer primitivo vira `str`; `int/real/bool` sao intercambiaveis; `str` nao converte para numero.

Conceito de Compiladores: Tratamento de funcoes embutidas (built-ins) na analise semantica.

Como explicar: As funcoes "de fabrica" (entrada/saida e conversoes) tem regras proprias, que ficam aqui.

InfoSemantica *(semantico)*

Responsabilidade. Empacota o **resultado** da analise para o back-end: a arvore **anotada** com tipos, mais as tabelas de funcoes e classes.

- E o "gancho" entre front-end e back-end (gerador `Jasmin`), evitando refazer a analise;
- `tipoDe(no)` devolve o tipo de qualquer expressao; tambem expoe as funcoes e classes declaradas.

Conceito de Compiladores: Representacao intermediaria anotada: a ponte entre analise e geracao de codigo.

Como explicar: Depois de validar, eu entrego ao back-end um "mapa" com o tipo de cada expressao e a lista de funcoes/classes.

8. Orquestracao (Main e Compilador)

Compilador (*br.ufpi.jss*)

Responsabilidade. A **fachada** do front-end: executa o pipeline completo (lexico -> sintaxe -> semantica) e devolve a mensagem de resultado.

- Liga JSSLexer + JSSParser (com o OuvinteErroSintatico) ao AnalisadorSemantico;
- Retorna Linguagem compilada com sucesso ou Erro na linha X: ...; e independente de stdin/stdout para poder ser testado.

Conceito de Compiladores: Orquestracao das fases do compilador (o "driver").

Como explicar: E o maestro que chama as tres fases na ordem certa e devolve o veredito.

Main (*br.ufpi.jss*)

Responsabilidade. O **ponto de entrada** do programa: le o codigo da entrada padrao e imprime o resultado.

- Le stdin como UTF-8 e **descarta um eventual BOM** inicial (arquivos salvos com BOM, ou o pipe do PowerShell);
- Imprime em UTF-8 (acentos corretos) e encerra com codigo 0 (sucesso) ou 1 (erro).

Conceito de Compiladores: Interface de linha de comando do compilador (item c da avaliacao).

Como explicar: E o "main" de verdade: pega o que vem pela entrada padrao e mostra a resposta.

9. Pacote codegen (gancho do back-end Jasmin)

GeradorJasmin (*codegen*)

Responsabilidade. O **esqueleto** do gerador de código intermediário **Jasmin** (back-end). Ainda **não implementado**.

- Estende JSSBaseVisitor<Void> e recebe a InfoSemantica produzida pelo front-end;
- Documenta o plano: emitir um arquivo .j, montado com jasmin.jar para gerar um .class executável na JVM.

Conceito de Compiladores: Geracao de código intermediário (síntese / back-end) - a próxima etapa do projeto.

Como explicar: É o lugar reservado para transformar o programa já validado em bytecode da JVM, via Jasmin.

10. Mapa: classe -> fase do compilador

Fase / conceito	Onde esta no código
Análise léxica (texto -> tokens)	JSS.g4 (regras léxicas) -> JSSLexer (gerado)
Análise sintática (tokens -> árvore)	JSS.g4 (regras de parser) -> JSSParser (gerado, ALL(*))
Tratamento de erros (parar no 1o)	erro/ErroCompilacao, erro/OuvinteErroSintatico
Sistema de tipos	tipos/ (Tipo, TipoPrimitivo, TipoVetor, TipoClasse, Tipos)
Tabela de símbolos e escopo	simbolos/ (Simbolo, SimboloFuncao, SimboloClasse, Escopo, TabelaDeSimbolos)
Análise semântica (type checking)	semantico/ (AnalizadorSemantico, RegrasOperadores, ValidadorNativas)
Representação intermediária anotada	semantico/InfoSemantica
Orquestração / linha de comando	Compilador, Main
Geração de código (back-end, futuro)	codegen/GeradorJasmin

11. Roteiro de apresentacao

Uma ordem sugerida para apresentar ao professor:

- **Visao geral.** Mostre o pipeline das tres fases e que o ANTLR gera o lexer e o parser a partir da gramatica.
- **Gramatica.** Abra a JSS.g4: mostre alguns tokens e a regra expr, explicando que a **ordem** das alternativas define a precedencia.
- **Demonstracao.** Rode um exemplo valido e um com erro (use exemplos/ok e exemplos/erros), mostrando a linha exata do erro.
- **Erros.** Explique o OuvinteErroSintatico e a regra de **parar no primeiro erro**.
- **Simbolos e tipos.** Mostre a TabelaDeSimbolos (escopo e shadowing) e o pacote tipos (ex.: `int->real` permitido, `real->int` nao).
- **Semantica.** Apresente o AnalisadorSemantico como o Visitor que une tudo; use 1-2 regras (ex.: o + e a atribuicao com promocao numerica).
- **Back-end.** Feche mostrando os ganchos: InfoSemantica (arvore anotada) e o esqueleto GeradorJasmin.

Deciso es para destacar: parar no **primeiro erro** com a linha; suporte a **vetores multidimensionais** (`int[2][3]`); `console.log` reconhecido **sem** tornar `console/log` palavras reservadas; `main` **opcional** e sem parametros; `float` nao existe (o correto e `real`).